[Logo oficial UMG — insertar]

UNIVERSIDAD MARIANO GÁLVEZ DE GUATEMALA

Maestría en Seguridad Informática

Curso: Seguridad en Aplicaciones

Laboratorio de Seguridad de Aplicaciones sobre OWASP Juice Shop

Fase 4: Implementación de Ciberdefensa

Fase 5: Despliegue Seguro e Infraestructura Cloud

Integrantes:

Molina Pérez, Juan Carlos

Sánchez Flores, Juan Walter Ernesto

García Gonzalez, Leopoldo Emanuel

Cambara Castillo, Vanessa Odeth

Catedrático: Ing. Matthew Alexander Seigne Samayoa

Guatemala, noviembre de 2026

Índice

Introducción....3
Objetivos....3
Fase 4: Implementación de Ciberdefensa....5
4.1 Pruebas de penetración – Caja negra (Black Box)....5
4.2 Pruebas basadas en riesgos – Caja gris / blanca....7
4.3 Modelado y ejecución de casos de abuso (Abuse Cases)....9
4.4 Delimitación del perímetro del software....11
Fase 5: Despliegue Seguro e Infraestructura Cloud....13
5.1 Escaneo de contenedores y artefactos....13
5.2 Hardening de Infraestructura como Código (IaC)....15
Delimitación de alcance: despliegue cloud diferido....20
Conclusiones....20
Referencias....21
Anexos....21

Introducción

El presente documento recopila las evidencias de las Fases 4 y 5 del proyecto de laboratorio de Seguridad de Aplicaciones, desarrollado sobre OWASP Juice Shop versión 20.3.0, una aplicación web deliberadamente vulnerable mantenida por la Open Worldwide Application Security Project (OWASP). La Fase 4 corresponde a la implementación de ciberdefensa mediante pruebas ofensivas y defensivas que simulan un ataque real, mientras que la Fase 5 aborda el despliegue seguro y la seguridad de la infraestructura (contenedores e Infraestructura como Código).

Todas las pruebas se ejecutaron bajo estrictas reglas de aislamiento: la instancia vulnerable únicamente escuchó en la dirección de bucle local (127.0.0.1) o dentro del contenedor efímero de integración continua, sin exponerse a Internet, y solo se evaluaron instancias propias del laboratorio. Cada hallazgo se respalda con evidencia verificable (capturas, reportes de herramientas y registros), cuyo valor hash SHA-256 se conserva en el manifiesto del repositorio, garantizando trazabilidad y la ausencia de resultados inventados.

Repositorio del proyecto: https://github.com/jmolinap1-universidad/juice-shop-lab

Objetivos

Objetivo general

Evaluar la postura de seguridad de una aplicación web real mediante pruebas de penetración y análisis de su infraestructura de despliegue, documentando con evidencia verificable las vulnerabilidades identificadas y los controles de aseguramiento aplicados.

Objetivos específicos

- Ejecutar pruebas de penetración de caja negra, gris y blanca sobre la aplicación, identificando vulnerabilidades explotables.
- Modelar y ejecutar casos de abuso que representen el uso malintencionado de funciones legítimas.
- Delimitar la superficie de ataque y el perímetro del software.

- Analizar las imágenes de contenedor en busca de vulnerabilidades del sistema operativo base y de las dependencias.
- Auditar la Infraestructura como Código (Dockerfiles y Terraform) para evitar configuraciones inseguras.

Fase 4: Implementación de Ciberdefensa

La Fase 4 comprende la realización de pruebas ofensivas y defensivas simulando un ataque real. Se emplearon técnicas de caja negra (sin conocimiento previo), caja gris y blanca (con acceso a arquitectura y código), la ejecución de casos de abuso y la delimitación del perímetro del software. La herramienta principal de automatización fue un conjunto de scripts propios ejecutados con cURL contra la instancia local, complementados con OWASP ZAP y Nmap.

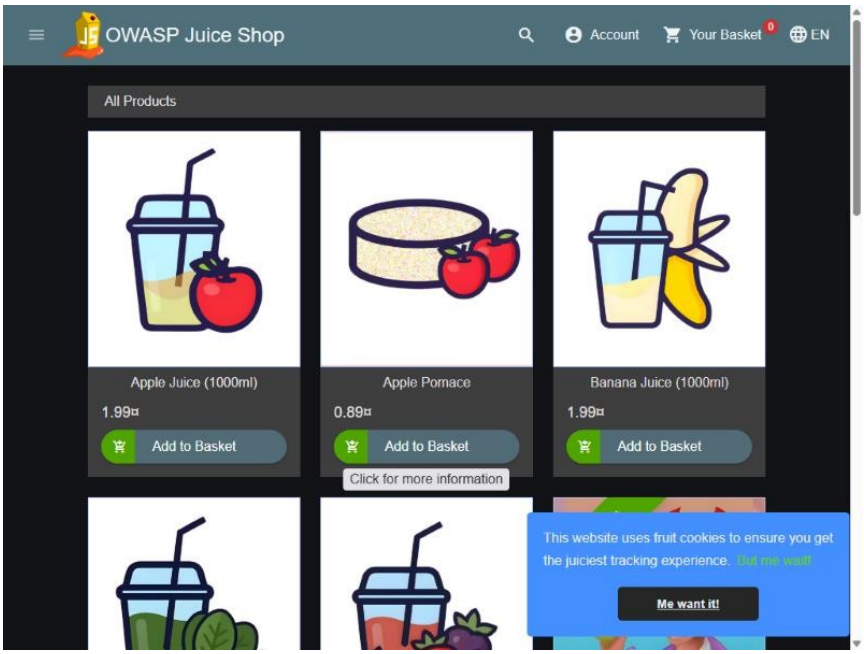


Figura 1. *Aplicación OWASP Juice Shop v20.3.0 en ejecución en el laboratorio local (http://127.0.0.1:3000).*

4.1 Pruebas de penetración – Caja negra (Black Box)

Las pruebas de caja negra simulan a un atacante externo sin conocimiento del código ni credenciales. Se realizó reconocimiento de la superficie expuesta con Nmap y un análisis dinámico pasivo con OWASP ZAP (perfil baseline), además de intentos de explotación a ciegas.

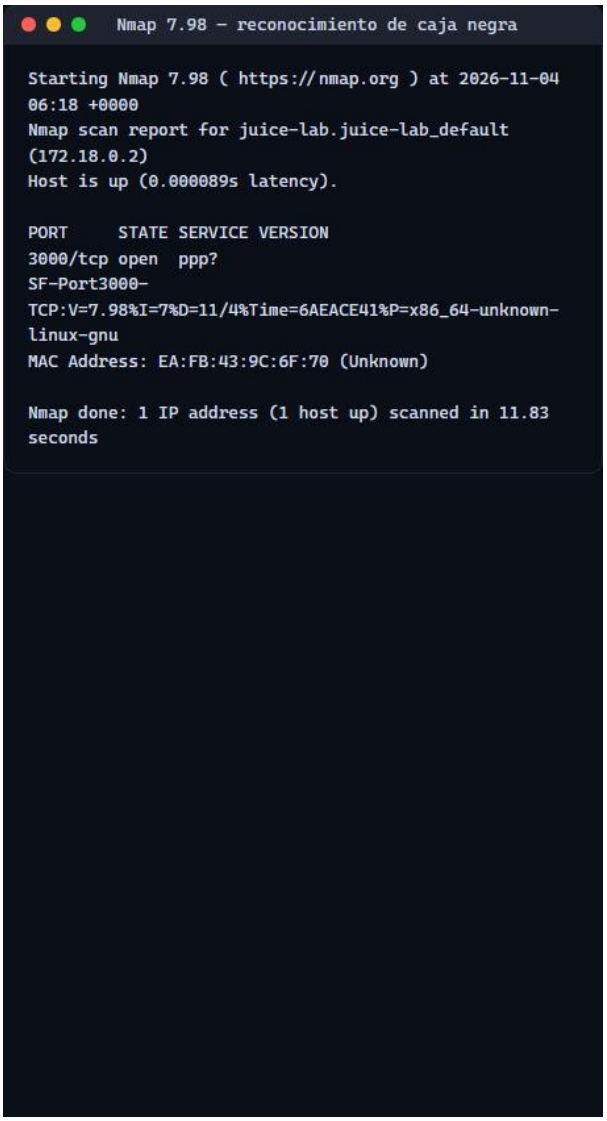
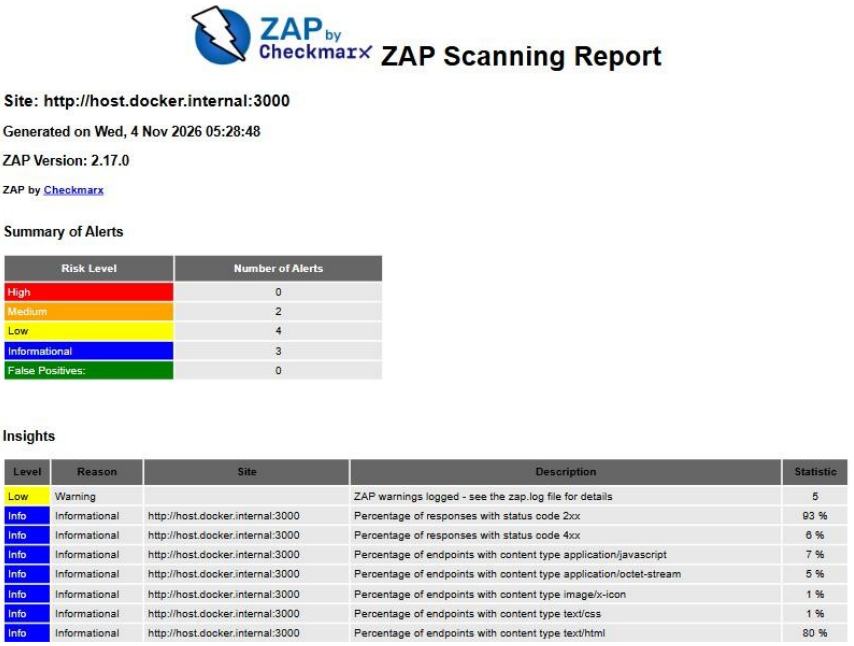


Figura 2. *Reconocimiento con Nmap 7.98: el puerto 3000/tcp se encuentra abierto; el banner revela el servicio HTTP y la cabecera permisiva Access-Control-Allow-Origin: *.*



ZAP by Checkmarx ZAP Scanning Report

Site: http://host.docker.internal:3000

Generated on Wed, 4 Nov 2026 05:28:48

ZAP Version: 2.17.0

ZAP by Checkmarx

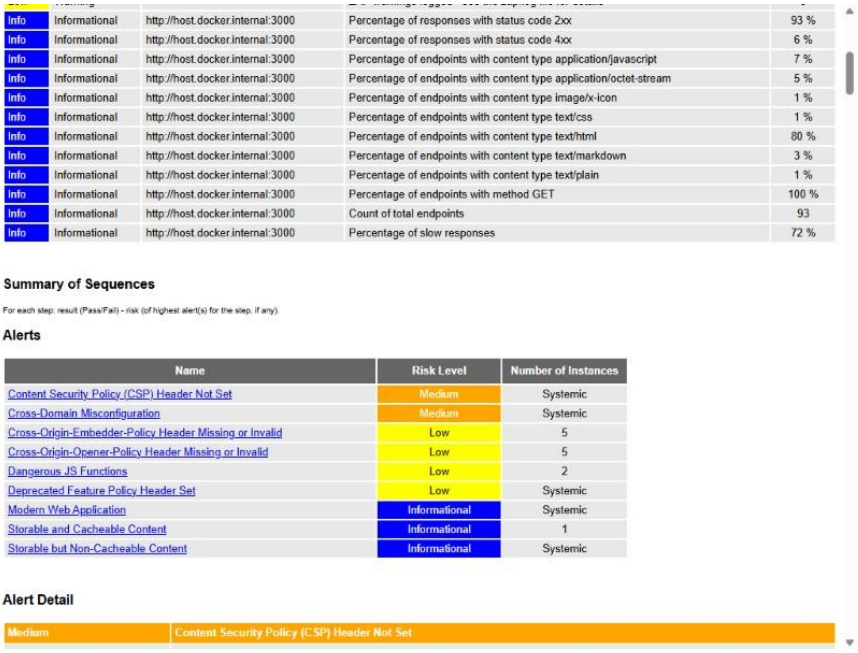
Summary of Alerts

Risk Level	Number of Alerts
High	0
Medium	2
Low	4
Informational	3
False Positives:	0

Insights

Level	Reason	Site	Description	Statistic
Low	Warning		ZAP warnings logged - see the zap.log file for details	5
Info	Informational	http://host.docker.internal:3000	Percentage of responses with status code 2xx	93 %
Info	Informational	http://host.docker.internal:3000	Percentage of responses with status code 4xx	6 %
Info	Informational	http://host.docker.internal:3000	Percentage of endpoints with content type application/javascript	7 %
Info	Informational	http://host.docker.internal:3000	Percentage of endpoints with content type application/octet-stream	5 %
Info	Informational	http://host.docker.internal:3000	Percentage of endpoints with content type image/x-icon	1 %
Info	Informational	http://host.docker.internal:3000	Percentage of endpoints with content type text/css	1 %
Info	Informational	http://host.docker.internal:3000	Percentage of endpoints with content type text/html	80 %

Figura 3. *Resumen del reporte de OWASP ZAP 2.17.0 (baseline): 2 alertas de riesgo medio, 4 bajas y 3 informativas, 0 falsos positivos.*



Info	Informational	http://host.docker.internal:3000	Percentage of responses with status code 2xx	93 %
Info	Informational	http://host.docker.internal:3000	Percentage of responses with status code 4xx	6 %
Info	Informational	http://host.docker.internal:3000	Percentage of endpoints with content type application/javascript	7 %
Info	Informational	http://host.docker.internal:3000	Percentage of endpoints with content type application/octet-stream	5 %
Info	Informational	http://host.docker.internal:3000	Percentage of endpoints with content type image/x-icon	1 %
Info	Informational	http://host.docker.internal:3000	Percentage of endpoints with content type text/css	1 %
Info	Informational	http://host.docker.internal:3000	Percentage of endpoints with content type text/html	80 %
Info	Informational	http://host.docker.internal:3000	Percentage of endpoints with content type text/markdown	3 %
Info	Informational	http://host.docker.internal:3000	Percentage of endpoints with content type text/plain	1 %
Info	Informational	http://host.docker.internal:3000	Percentage of endpoints with method GET	100 %
Info	Informational	http://host.docker.internal:3000	Count of total endpoints	93
Info	Informational	http://host.docker.internal:3000	Percentage of slow responses	72 %

Summary of Sequences

For each step: result (Pass/Fail) - risk (of highest alert(s) for the step, if any).

Alerts

Name	Risk Level	Number of Instances
Content Security Policy (CSP) Header Not Set	Medium	Systemic
Cross-Domain Misconfiguration	Medium	Systemic
Cross-Origin-Embedder-Policy Header Missing or Invalid	Low	5
Cross-Origin-Opener-Policy Header Missing or Invalid	Low	5
Dangerous JS Functions	Low	2
Deprecated Feature Policy Header Set	Low	Systemic
Modern Web Application	Informational	Systemic
Storable and Cacheable Content	Informational	1
Storable but Non-Cacheable Content	Informational	Systemic

Alert Detail

Medium	Content Security Policy (CSP) Header Not Set

Figura 4. *Detalle de alertas de ZAP: ausencia de Content Security Policy (CSP) y configuración incorrecta de CORS (Cross-Domain Misconfiguration), entre otras.*

4.2 Pruebas basadas en riesgos – Caja gris / blanca (Gray/White Box)

Con acceso a la arquitectura, las API y el código fuente, se verificó la lógica de autenticación y el control de acceso. Se confirmó una inyección SQL en el inicio de sesión (routes/login.ts) que permite autenticarse como administrador sin credenciales, y una

inyección SQL de tipo UNION en la búsqueda (routes/search.ts) que extrae la tabla de usuarios completa, incluyendo correos y hashes de contraseñas MD5.

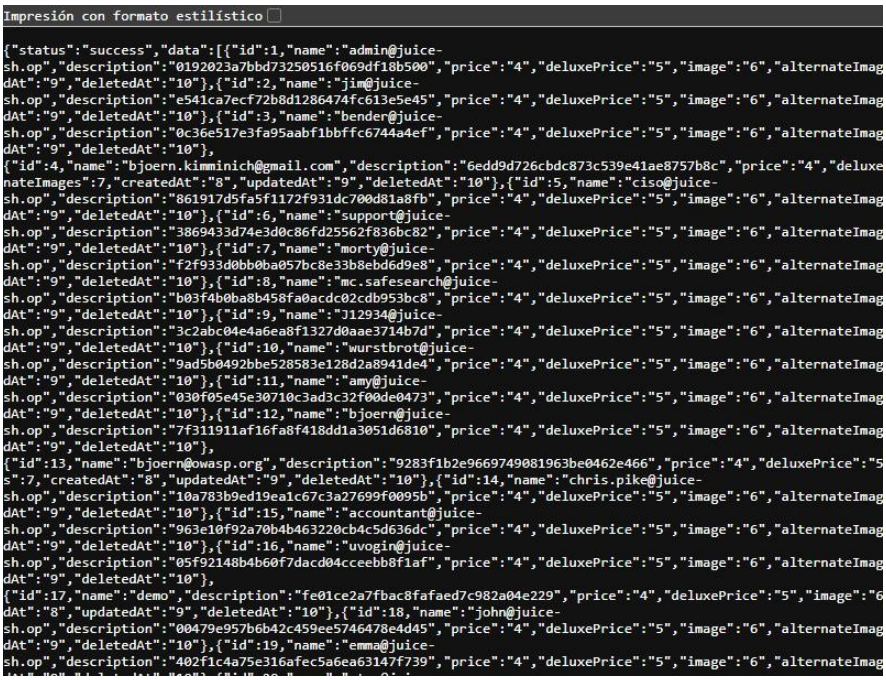


Figura 5. *Inyección SQL UNION en la búsqueda: la respuesta expone los correos y los hashes MD5 de todas las cuentas (p. ej. admin@juice-sh.op).*

Control de acceso: dos hallazgos *distintos*. Es importante no confundir dos debilidades diferentes de control de acceso. La **asignación masiva** (mass assignment, CWE-915) permite que el registro acepte el parámetro no filtrado "role": "admin" y crea una cuenta privilegiada. En cambio, el **BOLA/IDOR** (Broken Object Level Authorization, CWE-639) es la falta de verificación de propiedad de un objeto entre usuarios distintos.

Para demostrar un BOLA genuino se crearon dos cuentas propias de laboratorio con datos ficticios (A, víctima; B, atacante). Autenticado como B, se solicitó la cesta de compras de A mediante GET /rest/basket/{id de A} empleando el token de B. La aplicación respondió HTTP 200 devolviendo la cesta de A, demostrando que no verifica la propiedad del objeto. La prueba se contrastó con un control negativo (B accede a su propia cesta).

Campo	Valor observado
Cuenta A (víctima)	cesta id 6 — UserId 26
Cuenta B (atacante)	cesta id 7 — UserId 27
Petición	GET /rest/basket/6 con el token de B

Campo	Valor observado
Esperado (si fuera seguro)	HTTP 401/403, o solo la cesta propia de B
Observado	HTTP 200 devolviendo la cesta de A (UserId 26 ≠ 27)
Clasificación	VULNERABLE — BOLA/IDOR (CWE-639, OWASP API1:2023 / A01)

Evidencia: evidence/fase-4/bola/ (resultado-bola.md, cross-read.json, own-read.json). Limitación: la prueba demuestra la lectura de la cesta por identificador; no se generaliza a todos los objetos de la API.

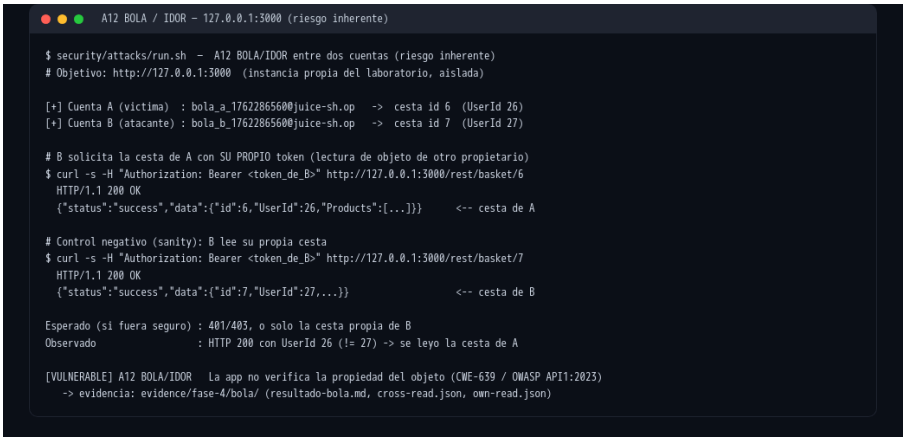


Figura 6. *Prueba BOLA/IDOR con dos cuentas: el usuario B solicita la cesta del usuario A (/rest/basket/6) con su propio token y la aplicación responde HTTP 200 devolviendo la cesta de A (UserId 26, distinto del 27 de B). Datos ficticios del laboratorio.*

4.3 Modelado y ejecución de casos de abuso (Abuse Cases)

Se definieron y ejecutaron escenarios donde un atacante abusa de funciones legítimas. El conjunto automatizado security/attacks/run.sh ejecutó once casos contra la instancia local. Para no sobreafirmar los resultados, cada observación se clasifica en una de cuatro clases: explotación demostrada, exposición observada, configuración insegura o ausencia de control. El resultado es: 4 vulnerabilidades explotadas (dos inyecciones SQL que comprometen la base de datos, enumeración de esquema y asignación masiva de rol), 4 exposiciones de información (listado de directorios, documento interno, métricas y clave pública), 2 configuraciones inseguras (CORS permisivo y ausencia de cabeceras) y 1 ausencia de control (sin limitación de intentos de inicio de sesión: 12 intentos con respuesta 401 y sin 429, lo que evidencia la falta de bloqueo pero no demuestra que se adivinara contraseña alguna).

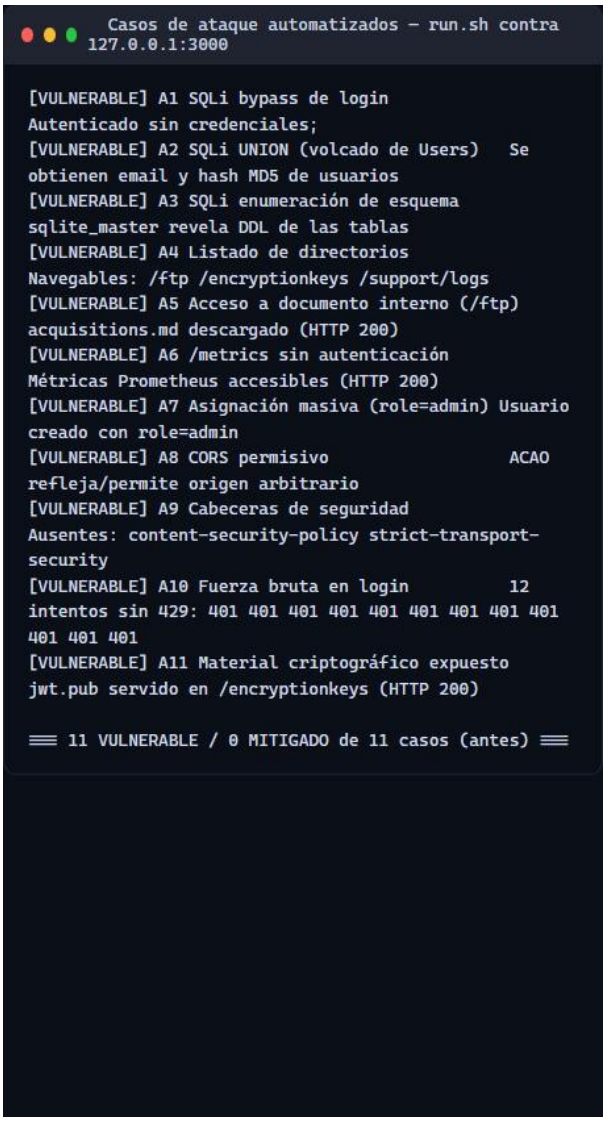


Figura 7. *Salida del conjunto de casos de ataque: las 11 pruebas registran una debilidad cada una; 4 constituyen explotación demostrada y 7 son exposiciones, configuraciones inseguras o ausencia de controles (riesgo inherente).*

Como evidencia complementaria de casos de abuso sobre funciones legítimas, la Figura 8 muestra el acceso sin autenticación al endpoint de métricas Prometheus (/metrics), que expone telemetría interna de la aplicación.

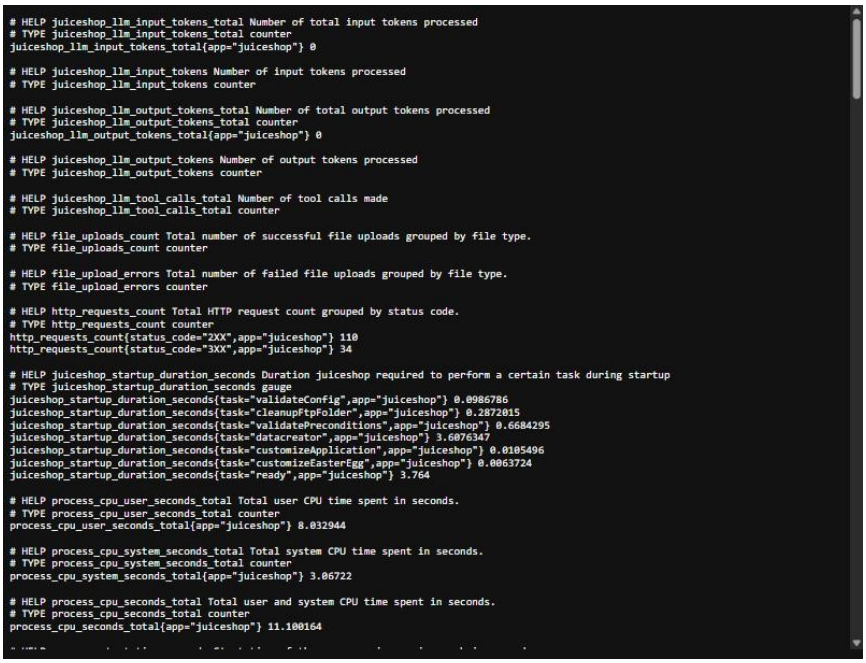


Figura 8. *Exfiltración de información: el endpoint /metrics se sirve sin autenticación, revelando métricas internas (conteos de peticiones, tiempos de arranque, CPU).*

4.4 Delimitación del perímetro del software

Se mapeó la superficie de ataque expuesta a partir del código con un conteo reproducible (comando grep sobre app/server.ts): 109 rutas declaradas (50 GET, 40 POST, 13 PUT, 5 DELETE, 1 PATCH; la suma coincide: 50+40+13+5+1 = 109), aproximadamente 81 endpoints bajo /rest, /api y /b2b, y 14 modelos de datos expuestos como API CRUD automática. Se identificaron puntos de entrada de archivos (carga de ZIP/XML/YAML, imagen por URL con riesgo de SSRF) y rutas estáticas con listado de directorios habilitado, que exponen documentos internos y material criptográfico. No se identificaron webhooks entrantes ni salientes (no aplica). Dado que la aplicación solo escuchó en 127.0.0.1, el perímetro evaluado es local y no un servicio publicado en Internet.

Elemento del perímetro	Detalle	Riesgo asociado
Listado de directorios	/ftp, /encryptionkeys, /support/logs	Exposición de documentos y claves (CWE-548)
Endpoint de diagnóstico	/metrics (Prometheus, sin autenticación)	Divulgación de información (CWE-200)
Carga de archivos	/file-upload, /profile/image/url, /rest/memories	XXE, SSRF (CWE-611, CWE-918)
API CRUD automática	14 modelos vía finale.resource	Control de acceso roto / IDOR (CWE-639)

Elemento del perímetro	Detalle	Riesgo asociado
Autenticación	JWT RS256 con clave privada embebida	Falsificación de tokens (CWE-798)

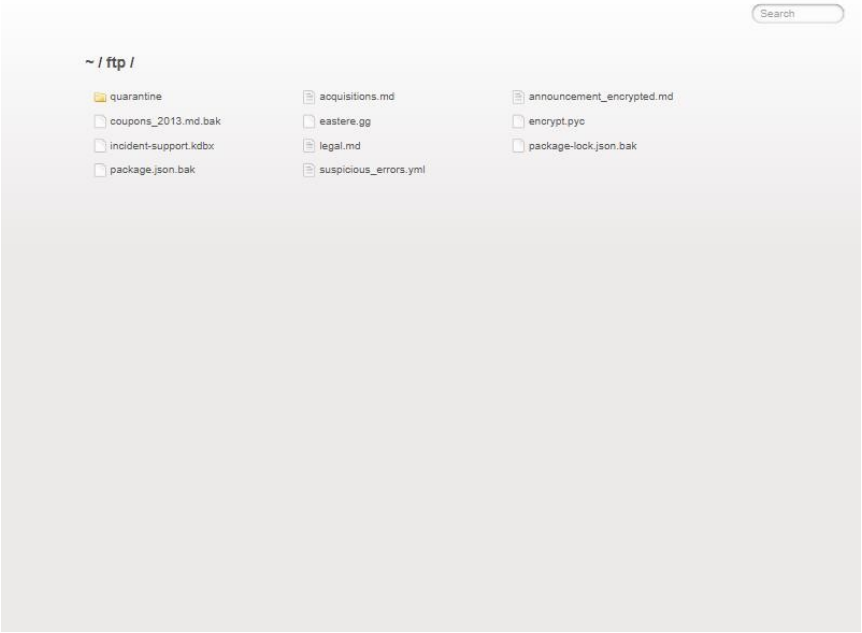


Figura 9. *Listado de directorios habilitado en /ftp: documentos internos accesibles (acquisitions.md, coupons_2013.md.bak, incident-support.kdbx).*

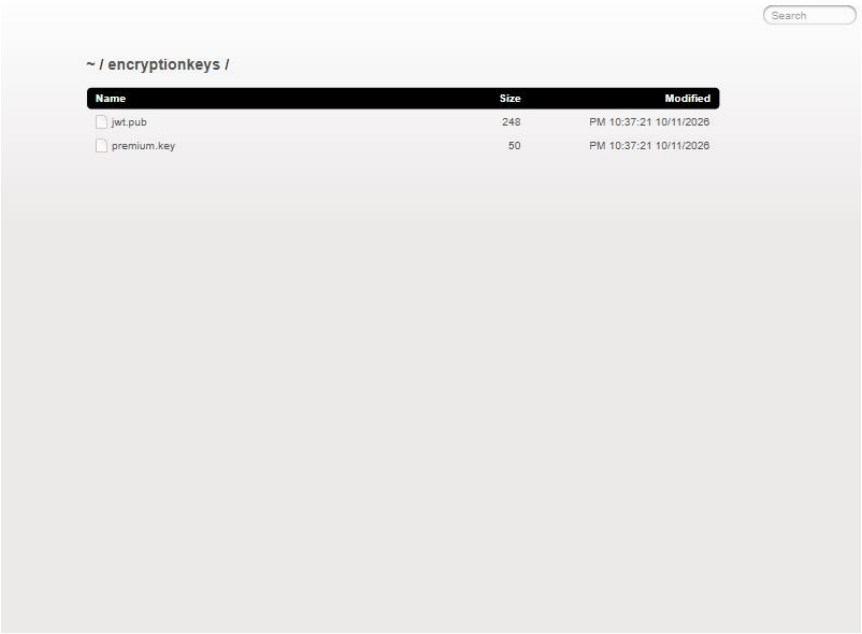


Figura 10. *Listado del directorio /encryptionkeys: se expone material criptográfico (jwt.pub, premium.key).*

Fase 5: Despliegue Seguro e Infraestructura Cloud (Release & CloudSec)

La Fase 5 evalúa la seguridad del despliegue: el escaneo de contenedores y artefactos en busca de vulnerabilidades, y el hardening de la Infraestructura como Código (IaC). Todas las herramientas se ejecutaron mediante Docker con versiones fijas, tanto en local como en el flujo de integración continua de GitHub Actions, garantizando reproducibilidad.

5.1 Escaneo de contenedores y artefactos

Se analizó la imagen oficial de Juice Shop (fijada por digest) con Trivy 0.75.0 y, de forma independiente, con Grype 0.120.1 sobre el SBOM generado con Syft (formato CycloneDX). Trivy identificó 11 vulnerabilidades críticas y 32 altas, además de 5 secretos y 66 malas configuraciones embebidas en los archivos de reto de la propia aplicación. La mayoría de las vulnerabilidades se concentran en las dependencias de Node.js (81) frente a las del sistema operativo base Debian 13 (45).



Figura 11. *Escaneo de la imagen con Trivy 0.75.0: 11 vulnerabilidades CRITICAL y 32 HIGH; detección de secretos y malas configuraciones.*



Figura 12. *Verificación independiente con Grype 0.120.1 sobre el SBOM (Syft): 12 críticas y 44 altas, confirmando los hallazgos de Trivy.*

Hallazgo relevante: el re-empaquetado o endurecimiento de la imagen no reduce las vulnerabilidades de las dependencias de la aplicación, pues residen en su código fuente; su corrección corresponde a la actualización de dependencias (documentada como trabajo propuesto, ya que rompe los retos de Juice Shop). El endurecimiento de la Fase 5 se centra en lo que sí es verificable: la higiene del Dockerfile y el endurecimiento del tiempo de ejecución del contenedor.

5.2 Hardening de Infraestructura como Código (IaC)

Se auditaron los Dockerfiles con Hadolint 2.15.1 y la Infraestructura como Código (Dockerfiles, Terraform y Compose) con Checkov 3.4.0. Es necesario distinguir dos

artefactos: el Dockerfile auditado app-hardened/Dockerfile (una propuesta de construcción desde el código fuente que corrige los 11 avisos del original y queda en 0, pero que no es la imagen desplegada) y el Dockerfile realmente desplegado deploy/hardened/Dockerfile, que produce la imagen juice-hardened:local a partir de la imagen oficial fijada por digest y cuyo endurecimiento proviene del tiempo de ejecución. Por tanto, la mejora de Hadolint pertenece al artefacto propuesto; la imagen en ejecución conserva las CVE de la imagen base, que se mitigan en runtime pero no se eliminan (su eliminación requiere recompilar con dependencias actualizadas, trabajo de la fase siguiente). Checkov validó 144 comprobaciones sin fallos en la arquitectura Terraform de la nube (con 5 omisiones justificadas) y 47 sin fallos en los Dockerfiles y archivos de despliegue.



Figura 13. *Hadolint: el Dockerfile original presenta 11 avisos (DL3059, DL3006); el Dockerfile endurecido queda en 0 avisos.*

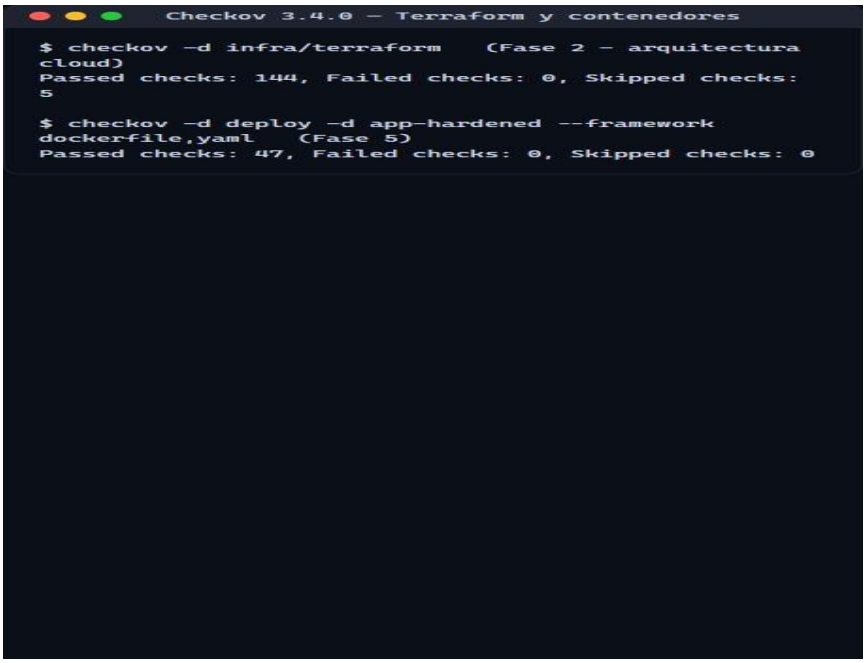


Figura 14. *Checkov 3.4.0: 144/0 en la arquitectura Terraform (Fase 2) y 47/0 en Dockerfiles y despliegue (Fase 5).*

Adicionalmente, se verificó el endurecimiento del tiempo de ejecución del contenedor mediante docker inspect: sistema de archivos raíz de solo lectura, eliminación de todas las capacidades de Linux (cap_drop ALL), prohibición de nuevos privilegios (no-new-privileges), usuario no root (65532), límites de procesos y memoria, y modo de producción. Se comprobó que la aplicación responde correctamente (HTTP 200) y que una escritura en el sistema de archivos raíz es bloqueada (EROFS).

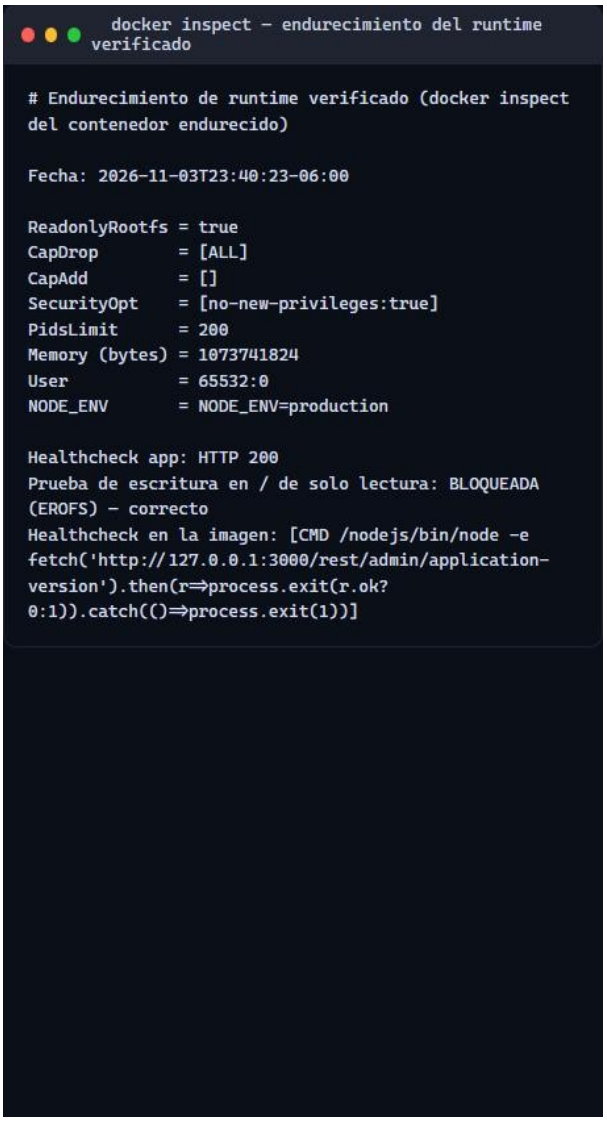


Figura 15. *Verificación del endurecimiento del runtime con docker inspect: FS de solo lectura, cap_drop ALL, no-new-privileges, límites y escritura bloqueada (EROFS).*

Delimitación de alcance: despliegue cloud diferido

Durante la entrega de las Fases 4 y 5, la evaluación práctica se realizó en una estación local mediante contenedores Docker y la ejecución controlada de herramientas de seguridad; la aplicación vulnerable no estuvo publicada en Internet. La infraestructura AWS se conserva como arquitectura objetivo definida en Terraform y sometida a análisis estático y validaciones de configuración (Checkov, terraform validate y plan), sin aplicar los recursos en una cuenta cloud. Esta separación permite obtener evidencias reproducibles de ciberdefensa, escaneo de imágenes y hardening de IaC, evitando costos y la exposición innecesaria de una aplicación deliberadamente vulnerable. El despliegue cloud y la verificación operativa de los controles administrados (ALB, WAFv2, RDS, API Gateway, CDN, KMS y SIEM) quedan planificados para la etapa final, previa autorización del equipo y aceptación del alcance por el catedrático.

Conclusiones

Las pruebas de la Fase 4 confirmaron un riesgo inherente alto: de las 11 observaciones registradas, 4 constituyen vulnerabilidades explotadas —incluido el compromiso total de la aplicación como administrador por dos vías independientes (inyección SQL y asignación masiva de rol)— y 7 corresponden a exposiciones de información, configuraciones inseguras o ausencia de controles. Adicionalmente, una prueba específica con dos cuentas confirmó un BOLA/IDOR genuino (lectura de la cesta de otro usuario). El reconocimiento de caja negra y el análisis de caja gris/blanca permitieron mapear con precisión la superficie de ataque y la causa raíz de cada hallazgo en el código, sin atribuir cobertura total del OWASP Top 10 ni afirmar más de lo demostrado.

La Fase 5 evidenció que las vulnerabilidades de las dependencias del contenedor no se resuelven con el empaquetado de la imagen, sino con la actualización del código; no obstante, el endurecimiento del Dockerfile (de 11 a 0 avisos de Hadolint), la auditoría de la Infraestructura como Código (Checkov 144/0 y 47/0) y el endurecimiento verificable del tiempo de ejecución reducen de forma medible la superficie de ataque del host y del contenedor.

En conjunto, el trabajo demuestra la aplicación de un enfoque DevSecOps con evidencia reproducible y trazable, sentando la base para la fase de mitigación (WAF, correcciones de código y medición del riesgo residual).

Referencias

OWASP Foundation. (2026). OWASP Juice Shop (Versión 20.3.0) [Software]. https://owasp-juice.shop

OWASP Foundation. (2025). OWASP Top 10:2025. https://owasp.org/Top10/

OWASP Foundation. (2025). OWASP Application Security Verification Standard (ASVS) 5.0.0. https://owasp.org/www-project-application-security-verification-standard/

Aqua Security. (2026). Trivy (Versión 0.75.0) [Software]. https://trivy.dev

Anchore. (2026). Grype y Syft [Software]. https://github.com/anchore

Checkov by Prisma Cloud. (2026). Checkov (Versión 3.4.0) [Software]. https://www.checkov.io

hadolint. (2026). Hadolint (Versión 2.15.1) [Software]. https://github.com/hadolint/hadolint

Nmap Project. (2026). Nmap (Versión 7.98) [Software]. https://nmap.org

OWASP Foundation. (2026). OWASP Zed Attack Proxy (ZAP) (Versión 2.17.0) [Software]. https://www.zaproxy.org

Anexos

Toda la evidencia (reportes JSON/SARIF/HTML, capturas y registros) está versionada en el repositorio del proyecto, con su valor hash SHA-256 en evidence/MANIFEST.sha256 y una entrada por comando en evidence/bitacora.md. Los entornos se reproducen con: make lab-up (laboratorio vulnerable) y make hard-up (entorno endurecido).